

Modelos de Parcial – Aprendizaje por Refuerzo

Inteligencia Artificial y Neurociencias
Universidad Torcuato Di Tella

Formato. Cada modelo combina **ejercicios prácticos** (seguimiento manual de algoritmos, cálculo de retornos y valores) con **preguntas de interpretación** sencillas sobre las fórmulas y los conceptos. Los temas cubiertos son: bandidos de k brazos, Procesos de Decisión de Markov, métodos Monte Carlo, métodos de Diferencias Temporales (SARSA y Q-learning), Deep Q-Learning y temas avanzados. Debajo de cada consigna se da una *respuesta posible* a modo de guía.

Modelo 1

1. Bandidos de k brazos (práctico)

Se aplica un algoritmo de bandidos de $k = 3$ brazos (acciones 1, 2, 3) con selección ε -greedy, estimando los valores con el promedio muestral y con $Q_1(a) = 0$ para toda a . La secuencia de acciones y recompensas es: $A_1=1, R_1=2$; $A_2=2, R_2=-1$; $A_3=1, R_3=1$; $A_4=3, R_4=2$; $A_5=1, R_5=0$. ¿En qué pasos seguro ocurrió el caso ε (acción al azar) y en qué pasos podría haber ocurrido?

Respuesta posible. Se calcula $Q_t(a)$ antes de cada paso y se compara la acción tomada con la(s) acción(es) greedy. Si la acción tomada *no* es greedy, seguro fue exploración; si lo es, pudo ser greedy o exploración.

t	$Q_t(1), Q_t(2), Q_t(3)$	greedy	A_t	conclusión
1	(0, 0, 0)	{1, 2, 3}	1	podría (empate)
2	(2, 0, 0)	{1}	2	seguro ε
3	(2, -1, 0)	{1}	1	podría
4	(1, 5, -1, 0)	{1}	3	seguro ε
5	(1, 5, -1, 2)	{3}	1	seguro ε

(Actualizaciones: tras $t=1$, $Q(1)=2$; tras $t=3$, $Q(1)=\frac{2+1}{2}=1,5$; tras $t=4$, $Q(3)=2$.) Entonces: **seguro** en $t = 2, 4, 5$; **podría** en $t = 1, 3$.

2. Bandidos (interpretación)

La regla de actualización incremental puede escribirse como $\text{NuevaEst} = \text{ViejaEst} + \text{TasaApr} [\text{Objetivo} - \text{ViejaEst}]$. Identifique qué es cada término en el caso del promedio muestral ($Q_{n+1} = Q_n + \frac{1}{n}[R_n - Q_n]$) y explique por qué en problemas no estacionarios conviene usar una tasa α constante en vez de $\frac{1}{n}$.

Respuesta posible. **ViejaEst** es Q_n (la estimación previa del valor de la acción), **NuevaEst** es Q_{n+1} , el **Objetivo** es la recompensa recién observada R_n , y la **TasaApr** es $\frac{1}{n}$. El término $[R_n - Q_n]$ es el *error de estimación*: la actualización mueve la estimación una fracción de ese error hacia el objetivo.

Con $\frac{1}{n}$, la tasa decrece con el tiempo y todas las recompensas pesan igual (promedio muestral), lo que converge al valor real *si* el valor no cambia. Pero si el problema es **no estacionario** (los valores $q_*(a)$ varían en el tiempo), interesa dar **más peso a las recompensas recientes**. Con α constante, $Q_{n+1} = (1 - \alpha)Q_n + \alpha R_n$: los pesos de las recompensas viejas decaen exponencialmente, así la estimación “olvida” el pasado y sigue los cambios del ambiente.

3. MDP: retornos (práctico)

Dada la secuencia de recompensas $R_1=2, R_2=0, R_3=-1, R_4=3, R_5=1$ con $T = 5$ y $\gamma = 0,5$, calcular $G_0, G_1, \dots, G_4$. *Ayuda: de atrás hacia adelante usando $G_t = R_{t+1} + \gamma G_{t+1}$.*

Respuesta posible. Con $G_5 = 0$:

$$\begin{aligned} G_4 &= R_5 + \gamma G_5 = 1 + 0,5 \cdot 0 = 1 \\ G_3 &= R_4 + \gamma G_4 = 3 + 0,5 \cdot 1 = 3,5 \\ G_2 &= R_3 + \gamma G_3 = -1 + 0,5 \cdot 3,5 = 0,75 \\ G_1 &= R_2 + \gamma G_2 = 0 + 0,5 \cdot 0,75 = 0,375 \\ G_0 &= R_1 + \gamma G_1 = 2 + 0,5 \cdot 0,375 = 2,1875 \end{aligned}$$

4. MDP (interpretación)

Explique qué representa el factor de descuento γ en $G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k$. ¿Qué comportamiento induce $\gamma = 0$ y qué comportamiento induce $\gamma \rightarrow 1$? Enuncie además la propiedad de Markov.

Respuesta posible. $\gamma \in [0, 1]$ determina cuánto valora el agente las recompensas futuras frente a las inmediatas. Con $\gamma = 0$ el agente es *miope*: solo le importa la recompensa inmediata R_{t+1} . Con $\gamma \rightarrow 1$ valora casi por igual recompensas lejanas y cercanas (agente “previsor”). Sirve la analogía con la inflación: una inflación alta (que licua el valor del dinero futuro) equivale a un γ bajo.

La **propiedad de Markov** dice que la probabilidad del próximo estado y recompensa depende *solo* del estado y la acción actuales, y no de la historia previa: $p(s', r \mid s, a) = \Pr\{S_t=s', R_t=r \mid S_{t-1}=s, A_{t-1}=a\}$. Esto obliga a definir los estados de modo que incluyan toda la información del pasado relevante para el futuro.

5. Q-learning (práctico)

Ambiente con estados $\{s_1, s_2\}$ y acciones $\{\leftarrow, \rightarrow\}$ (tarea continua). Se ejecuta Q-learning con $\alpha = 0,5$, $\gamma = 0,5$ y Q inicializada en 0. La trayectoria es: $s_1, \rightarrow, +4, s_2, \rightarrow, +2, s_1, \rightarrow, +8, s_2, \dots$. **Hacer el seguimiento de Q tras cada acción y dar la política greedy resultante.**

Respuesta posible. Regla: $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$.

Paso 1 ($s_1, \rightarrow, +4, s_2$): $\max_a Q(s_2, a) = 0$. $Q(s_1, \rightarrow) = 0 + 0,5[4 + 0,5 \cdot 0 - 0] = 2$.

Paso 2 ($s_2, \rightarrow, +2, s_1$): $\max_a Q(s_1, a) = \max(2, 0) = 2$. $Q(s_2, \rightarrow) = 0 + 0,5[2 + 0,5 \cdot 2 - 0] = 1,5$.

Paso 3 ($s_1, \rightarrow, +8, s_2$): $\max_a Q(s_2, a) = \max(1,5, 0) = 1,5$. $Q(s_1, \rightarrow) = 2 + 0,5[8 + 0,5 \cdot 1,5 - 2] = 2 + 0,5 \cdot 6,75 = 5,375$.

Tabla final: $Q(s_1, \rightarrow) = 5,375$, $Q(s_2, \rightarrow) = 1,5$, resto = 0. **Política greedy**: en s_1 elegir $\rightarrow$; en s_2 elegir $\rightarrow$.

6. TD (interpretación)

Escriba las reglas de actualización de SARSA y Q-learning y explique cuál es la única diferencia entre ambas. ¿Por qué se dice que SARSA es *on-policy* y Q-learning *off-policy*?

Respuesta posible.

$$\text{SARSA: } Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$$

$$\text{Q-learning: } Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

La única diferencia está en cómo estiman el retorno desde S' : SARSA usa $Q(S', A')$, donde A' es la acción que **realmente** va a tomar según su política ε -greedy; Q-learning usa $\max_a Q(S', a)$, es decir asume una política **greedy pura** para estimar el retorno.

Por eso SARSA es **on-policy**: evalúa y mejora la misma política que usa para actuar (incluye el efecto de la exploración). Q-learning es **off-policy**: la política que estima (greedy) está *desacoplada* de la que sigue para explorar (ε -greedy).

Modelo 2

1. Bandidos: actualización incremental (práctico)

Un algoritmo de bandidos con promedio muestral tiene, para la acción a , la estimación $Q_n(a) = 3$ luego de haberla tomado $n - 1 = 4$ veces. Se vuelve a elegir a y se obtiene $R_n = 8$. (a) Calcular $Q_{n+1}(a)$ con la fórmula incremental. (b) ¿Por qué esta forma es preferible a guardar todas las recompensas y promediar?

Respuesta posible. (a) $n = 5$, entonces

$$Q_{n+1}(a) = Q_n + \frac{1}{n} [R_n - Q_n] = 3 + \frac{1}{5} [8 - 3] = 3 + 1 = 4.$$

(b) La actualización incremental requiere almacenar solo Q_n y n (memoria $O(1)$) y cada actualización cuesta $O(1)$, en vez de guardar la lista completa de recompensas y recomputar la media (memoria y tiempo crecientes con la cantidad de muestras).

2. Bandidos (interpretación)

¿Es equivalente usar una política ε -greedy con $\varepsilon = 0,1$ durante 1000 pasos a primero ejecutar 100 pasos puramente exploratorios y luego 900 pasos puramente greedy? Justifique. Explique también la idea de los valores iniciales optimistas como forma de exploración.

Respuesta posible. No son equivalentes. Con ε -greedy la exploración está **distribuida** a lo largo de todo el proceso: sigue explorando (aprox. un 10 % del tiempo) incluso al final, lo que le permite corregir estimaciones erróneas y adaptarse (útil si el problema no es estacionario). El esquema “explorar primero, explotar después” fija las estimaciones tras la fase inicial: si por azar una buena acción quedó subestimada en esos primeros pasos, ya nunca la vuelve a probar. Además, durante la fase greedy no obtiene más información de las acciones no elegidas.

Valores iniciales optimistas: se inicializa $Q_1(a)$ con un valor muy alto para toda a . Como toda recompensa real resulta “decepcionante” frente a la estimación inflada, el error $[R - Q]$ es negativo y el agente se ve *empujado* a probar las otras acciones (que siguen con valor alto). Así se logra una exploración inicial sistemática sin necesidad de aleatoriedad. Su limitación: es un mecanismo transitorio, poco útil en problemas no estacionarios.

3. MDP: funciones de valor (práctico)

Ambiente determinístico en línea: $A \rightarrow B \rightarrow C \rightarrow D$, con D terminal. **Recompensas:** $A \rightarrow B$ da 1, $B \rightarrow C$ da 2, $C \rightarrow D$ da 3. **Con la política π que avanza siempre a la derecha, calcular $v_\pi(A), v_\pi(B), v_\pi(C)$ para $\gamma = 0$, $\gamma = 0,5$ y $\gamma = 1$.**

Respuesta posible. Usando $v_\pi(s) = r + \gamma v_\pi(s')$ desde el final ($v_\pi(D) = 0$):

$$v_\pi(C) = 3 + \gamma \cdot 0 = 3, \quad v_\pi(B) = 2 + \gamma v_\pi(C), \quad v_\pi(A) = 1 + \gamma v_\pi(B).$$

γ	$v_\pi(A)$	$v_\pi(B)$	$v_\pi(C)$
0	1	2	3
0,5	2,75	3,5	3
1	6	5	3

(Con $\gamma = 0,5$: $v_\pi(B) = 2 + 0,5 \cdot 3 = 3,5$; $v_\pi(A) = 1 + 0,5 \cdot 3,5 = 2,75$. Con $\gamma = 1$: $v_\pi(B) = 5$, $v_\pi(A) = 6$.) Como π es determinística, $q_\pi(s, \rightarrow) = v_\pi(s)$ para la acción seguida.

4. Monte Carlo (práctico)

Ambiente con estados $\{s_1, s_2, s_3\}$ (s_3 terminal) y acciones $\{a, b\}$. Se corre Monte Carlo para estimar π_* con $\gamma = 1$ y Q inicializada en 0. Episodios: *Ep. 1:* $s_1, a, +2, s_2, a, +3, s_3$; *Ep. 2:* $s_1, a, +1, s_2, b, +4, s_3$. Indicar Retornos(S, A) y $Q(S, A)$ al final de cada episodio, y la acción greedy en cada estado al comenzar el tercer episodio.

Respuesta posible. Los retornos ($\gamma = 1$) son la suma de recompensas desde cada par (S, A) .

Fin Ep. 1: $G(s_1, a) = 2 + 3 = 5$, $G(s_2, a) = 3$. $\text{Ret}(s_1, a) = [5] \Rightarrow Q(s_1, a) = 5$; $\text{Ret}(s_2, a) = [3] \Rightarrow Q(s_2, a) = 3$.

Fin Ep. 2: $G(s_1, a) = 1 + 4 = 5$, $G(s_2, b) = 4$. $\text{Ret}(s_1, a) = [5, 5] \Rightarrow Q(s_1, a) = 5$; $\text{Ret}(s_2, b) = [4] \Rightarrow Q(s_2, b) = 4$.

Tabla: $Q(s_1, a) = 5$, $Q(s_1, b) = 0$, $Q(s_2, a) = 3$, $Q(s_2, b) = 4$. **Greedy** al comenzar Ep. 3: en s_1 elegir a ($5 > 0$); en s_2 elegir b ($4 > 3$).

5. MC vs TD (interpretación)

¿Cuál es la diferencia fundamental entre los métodos Monte Carlo y los de Diferencias Temporales en cuándo y cómo actualizan Q ? Defina bootstrapping y dé una situación donde TD tenga ventaja práctica sobre MC.

Respuesta posible. Monte Carlo espera al final del episodio y actualiza usando el retorno real medido G_t : $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G_t - Q(S_t, A_t)]$. TD actualiza tras cada paso, sin esperar el final, usando $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ como objetivo.

Bootstrapping: estimar un valor a partir de otras estimaciones propias (aquí, $Q(S_{t+1}, \cdot)$) en lugar de esperar el resultado real completo. TD hace bootstrapping; MC no.

Ventaja de TD: en tareas continuas (sin fin) o con episodios muy largos, MC no podría actualizar (no hay retorno final que medir), mientras que TD aprende “sobre la marcha”. Ejemplo: un agente en un videojuego con muchos estados aprende en tiempo real con TD, mientras que MC tendría que completar la partida entera para cada actualización.

6. Deep Q-Learning (interpretación)

¿Por qué los métodos tabulares no sirven en problemas como jugar al Atari desde los píxeles? ¿Qué se hace en su lugar? Explique brevemente para qué sirven el replay de experiencias y la target network en DQN.

Respuesta posible. Los métodos tabulares guardan un valor por cada par (s, a) en una tabla Q , lo cual es imposible cuando el espacio de estados $\mathcal{S}$ es gigantesco: la cantidad de pantallas posibles de un juego es combinatoriamente enorme, y además dos pantallas casi idénticas cuentan como estados distintos (no hay generalización). En su lugar se usa **aproximación de la función de valor**: se reemplaza la tabla por una función parametrizada $\hat{q}(s, a, \mathbf{w})$ (una red neuronal con pesos $\mathbf{w}$, $d \ll |\mathcal{S}|$) que se ajusta con descenso por gradiente (semi-gradiente), permitiendo generalizar a estados nunca vistos.

Replay de experiencias: se almacenan las transiciones $e_t = (S_t, A_t, R_{t+1}, S_{t+1})$ en una memoria y cada actualización se hace sobre un *minibatch* muestreado al azar; esto rompe la correlación temporal entre muestras consecutivas y estabiliza el entrenamiento. **Target network:** una copia de la red que se usa para computar el objetivo y se actualiza solo cada cierta cantidad de iteraciones; al mantener el objetivo “fijo” un rato, evita que la red persiga un blanco que se mueve con cada paso.

Modelo 3

1. Bandidos: UCB (práctico)

Se usa selección por límite de confianza superior (UCB): $A_t \doteq \arg \max_a [Q_t(a) + c\sqrt{\ln t / N_t(a)}]$, con $c = 1$. Ya se ejecutaron 4 pasos ($t = 5$, $\ln 5 \approx 1,609$) y el estado es $N(1)=2, Q(1)=1$; $N(2)=1, Q(2)=2$; $N(3)=1, Q(3)=0$. ¿Qué acción elige UCB en $t = 5$?

Respuesta posible.

$$\text{UCB}(1) = 1 + \sqrt{1,609/2} = 1 + 0,897 \approx 1,90$$

$$\text{UCB}(2) = 2 + \sqrt{1,609/1} = 2 + 1,269 \approx 3,27$$

$$\text{UCB}(3) = 0 + \sqrt{1,609/1} = 1,269 \approx 1,27$$

El máximo es la **acción 2**. (El término $c\sqrt{\ln t / N_t(a)}$ crece cuando una acción se exploró poco: premia la *incertidumbre*, favoreciendo acciones con pocas visitas.)

2. MDP: retorno infinito (práctico)

(a) Con $\gamma = 0,8$ y la secuencia $R_1 = 3$ seguida de una sucesión infinita de dieces ($R_2 = R_3 = \dots = 10$), calcular G_0 . (b) Demostrar que si $R_t = 1$ para todo t , entonces $G_t = \frac{1}{1-\gamma}$.

Respuesta posible. (a) Desde $t = 1$, la cola es una serie geométrica: $G_1 = \sum_{k=0}^{\infty} 10 \gamma^k = \frac{10}{1-0,8} = 50$. Entonces $G_0 = R_1 + \gamma G_1 = 3 + 0,8 \cdot 50 = \mathbf{43}$.

(b) $G_t = \sum_{k=0}^{\infty} \gamma^k \cdot 1 = 1 + \gamma + \gamma^2 + \dots = \frac{1}{1-\gamma}$ (serie geométrica de razón $\gamma < 1$). Alternativamente, por la forma recursiva $G_t = 1 + \gamma G_t \Rightarrow G_t(1 - \gamma) = 1 \Rightarrow G_t = \frac{1}{1-\gamma}$.

3. SARSA vs Q-learning en una transición (práctico)

La tabla Q tiene: $Q(0, a) = 4$; **en el estado 1:** $Q(1, x) = 6, Q(1, y) = -2, Q(1, z) = 1$. **El agente está en $S = 0$, ejecuta a , recibe $R = 3$ y pasa a $S' = 1$. Con $\gamma = 0,5$ y $\alpha = 0,5$:** (a) actualizar $Q(0, a)$ por **Q-learning**; (b) actualizar $Q(0, a)$ por **SARSA** considerando todas las opciones de A' (explotación y cada exploración posible).

Respuesta posible. (a) **Q-learning:** usa $\max_a Q(1, a) = 6$.

$$Q(0, a) \leftarrow 4 + 0,5[3 + 0,5 \cdot 6 - 4] = 4 + 0,5 \cdot 2 = \mathbf{5}.$$

(b) **SARSA:** depende de la acción A' que se elija en $S' = 1$.

- Explotación / exploración que cae en x ($Q = 6$): $4 + 0,5[3 + 0,5 \cdot 6 - 4] = \mathbf{5}$.
- Exploración en y ($Q = -2$): $4 + 0,5[3 + 0,5 \cdot (-2) - 4] = 4 + 0,5 \cdot (-2) = \mathbf{3}$.
- Exploración en z ($Q = 1$): $4 + 0,5[3 + 0,5 \cdot 1 - 4] = 4 + 0,5 \cdot (-0,5) = \mathbf{3,75}$.

Q-learning siempre toma el máximo; SARSA usa el valor de la acción realmente elegida, por eso su resultado varía con la exploración.

4. Q-learning: parámetros (interpretación)

En Q-learning, ¿qué efecto tiene usar valores extremos en cada parámetro? Analice: (a) $\alpha = 0$ y $\alpha = 1$; (b) $\gamma = 0$ y $\gamma = 1$; (c) $\varepsilon = 0$ y $\varepsilon = 1$.

Respuesta posible. (a) α (**tasa de aprendizaje**). Con $\alpha = 0$ la actualización es nula: Q nunca cambia, no aprende. Con $\alpha = 1$ la nueva estimación reemplaza por completo a la vieja ($Q(S, A) = R + \gamma \max_a Q(S', a)$): aprende solo de la última experiencia, útil en ambientes determinísticos pero inestable si hay ruido.

(b) γ (**descuento**). Con $\gamma = 0$ el agente es miope: solo maximiza la recompensa inmediata, ignora el futuro. Con $\gamma = 1$ pesa todas las recompensas futuras por igual; en tareas continuas el retorno puede diverger.

(c) ε (**exploración**). Con $\varepsilon = 0$ es puramente greedy: nunca explora, puede quedar atrapado en una política subóptima. Con $\varepsilon = 1$ actúa siempre al azar: explora todo pero nunca aprovecha lo aprendido (aunque, al ser off-policy, Q-learning igual puede estimar la política óptima si visita suficientemente todos los pares).

5. Ambiente estocástico vs determinístico (interpretación)

En el juego “Cuatro en línea” (dos jugadores), ¿la función $p(s', r \mid s, a)$ que caracteriza el ambiente es estocástica o determinística? Justifique. ¿Qué significa que $\sum_{s', r} p(s', r \mid s, a) = 1$?

Respuesta posible. Desde la perspectiva del agente, el ambiente incluye al **oponente**. Si el oponente no juega siempre lo mismo (tiene una política estocástica o desconocida), entonces tras la acción del agente el estado resultante s' (que depende de la respuesta del rival) no es único: el ambiente es **estocástico**. Si el oponente fuera totalmente determinístico y conocido, p sería determinística. La regla del juego en sí es determinística, pero la dinámica p vista por el agente hereda la incertidumbre del rival.

$\sum_{s', r} p(s', r \mid s, a) = 1$ expresa que, fijados un estado s y una acción a , la distribución sobre todos los posibles pares (estado siguiente, recompensa) es una distribución de probabilidad válida: *algo* ocurre con certeza, y las probabilidades de todos los resultados posibles suman 1.

6. Temas avanzados (interpretación)

Los métodos de gradiente de políticas (p.ej. REINFORCE) se contraponen a los basados en función de valor. ¿Qué aprenden directamente y en qué casos conviene usarlos? Explique además, en una línea, qué hace Monte Carlo Tree Search (MCTS).

Respuesta posible. Los métodos de gradiente de políticas aprenden **directamente una política parametrizada** $\pi(a \mid s, \theta)$ (las probabilidades de cada acción), sin pasar por una función de valor q_* . Se ajustan los parámetros por ascenso de gradiente sobre una métrica de desempeño $J(\theta)$. En REINFORCE, la probabilidad de una acción **sube o baja proporcionalmente al retorno observado** G . Conviene cuando hay **demasiadas acciones**, acciones **continuas**, o cuando es más simple representar la política que el valor (operación de brazos robóticos, trading, LLMs).

MCTS: planificación en tiempo de decisión que construye incrementalmente un árbol desde el estado actual, simulando trayectorias (selección–expansión–simulación–backup) para estimar el valor de las acciones y elegir la mejor. Es el motor de búsqueda de AlphaGo.

Banco de preguntas extra (para practicar)

Bandidos de k brazos

- Explique el compromiso (*trade-off*) entre **exploración** y **explotación**. ¿Qué es una acción *greedy*?
- Diferencia entre $q_*(a)$ (valor real) y $Q_t(a)$ (estimación). ¿Por qué $Q_t(a) \rightarrow q_*(a)$ con el promedio muestral? (Ley de los Grandes Números).
- En el ejemplo de 10 brazos, ¿por qué $\varepsilon = 0,1$ termina eligiendo más la acción óptima que $\varepsilon = 0$ (greedy puro)?

Procesos de Decisión de Markov

- Distinga $v_\pi(s)$ de $q_\pi(s, a)$ y escriba la relación $q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t=s, A_t=a]$.
- Exprese $p(s' \mid s, a)$ y $r(s, a)$ en función de $p(s', r \mid s, a)$ (marginalizando y tomando esperanza).
- Diferencia entre métodos **model-based** (p.ej. Value Iteration) y **model-free** (Monte Carlo, Q-learning). ¿Por qué casi siempre usamos model-free?
- ¿Qué es una política óptima π_* y las funciones óptimas v_*, q_* ?
- Modele el Tatetí como MDP: estados, acciones, recompensas, ¿es determinístico o estocástico?

Monte Carlo y Diferencias Temporales

- ¿Por qué el primer algoritmo MC para estimar π_* necesita una política ε -greedy y no una greedy pura? (Para visitar todos los pares (s, a)).
- Escriba la actualización incremental de la media de retornos: $Q \leftarrow Q + \frac{1}{k+1}(G^{(k+1)} - Q)$. ¿Con qué regla de bandidos se parece?
- Efecto de arrancar con ε alto (p.ej. 1) y bajarlo gradualmente hasta $\approx 0,01$ en MC. (Explorar mucho al principio, explotar al final).
- En el ejemplo “conduciendo a casa”, ¿por qué TD corrige la predicción en cada tramo y MC recién al llegar?

Deep Q-Learning y temas avanzados

- ¿Qué es el error cuadrático $\overline{QE}(\mathbf{w}) = \sum_{s,a} [q_*(s, a) - \hat{q}(s, a, \mathbf{w})]^2$ y cómo se lo reduce con SGD?
- ¿Por qué DQL se llama método *semi-gradiente* y por qué no garantiza convergencia (a diferencia del caso tabular)?
- Métodos **Actor-Critic**: ¿qué rol cumple el *actor* y qué rol el *crítico*? ¿En qué se diferencia de REINFORCE? (Bootstrapping / objetivo = error TD).
- Los cinco pasos de MCTS: selección, expansión, simulación, backup, repetir. Diferencia entre *política del árbol* y *política rollout*.

- **AlphaGo vs AlphaGo Zero:** ¿en qué se diferencian respecto del uso de datos humanos y del *self-play* (*tabula rasa*)?
- Preguntas para neurociencias: ¿tiene el cerebro un mecanismo parecido a ε -greedy? ¿Cómo se relaciona la recompensa de RL con la dopamina? ¿Hay algo análogo al *replay de experiencias* durante el sueño?